

Bulk Operations Performance Benchmark in Entity Framework Core

In this article, I will discuss **Performance Benchmarks Between Standard Entity Framework Core Bulk Operation vs. EFCore.BulkExtensions vs. Z.EntityFramework.Extensions.EFCore** with Examples. Please read our previous article discussing [Bulk Operations in Entity Framework Core using EFCore.BulkExtensions](#) with Examples.

Bulk Operations Performance Benchmark in Entity Framework Core

Entity Framework Core (EF Core) is a powerful Object-Relational Mapping (ORM) framework that allows developers to interact with databases using .NET objects. While EF Core simplifies data access, it may not always provide optimal performance for bulk operations involving large datasets.

Why Do We Need a Performance Benchmark in Entity Framework Core?

When working with large datasets, database operations' performance becomes crucial. Entity Framework Core can become inefficient when performing bulk operations like inserting, updating, or deleting thousands of records. To address these performance limitations, various libraries, such as EFCore.BulkExtensions and Z.EntityFramework.Extensions.EFCore, are available to optimize such operations.

Conducting a Performance Benchmark helps us compare the performance of the standard Entity Framework Core approach against these optimized libraries and decide which is the most suitable for our needs. Performance Benchmark in Entity Framework Core, measures the time it takes to execute bulk operations such as inserts, updates, and deletes across different libraries and approaches.

Standard EF Core, EFCore.BulkExtensions, and Z.EntityFramework.Extensions.EFCore Bulk Operations Performance Benchmark

Let's compare Standard EF Core, EFCore.BulkExtensions, and Z.EntityFramework.Extensions.EFCore in terms of performance during bulk operations in a .NET Console Application. The benchmark involves inserting, updating, and deleting 2000 records to understand the differences in execution times and determine which method performs best. This benchmark compares three methods of performing bulk operations:

- **Standard EF Core:** Uses EF Core's built-in methods (AddRange, UpdateRange, RemoveRange) combined with SaveChanges().
- **EFCore.BulkExtensions:** A third-party library that provides optimized bulk operation methods.
- **Z.EntityFramework.Extensions.EFCore:** Another third-party library offering enhanced bulk operation capabilities.

Please make sure to Install the following two Packages:

- **EFCore.BulkExtensions**
- **Z.EntityFramework.Extensions.EFCore**

Creating the Product Entity

To simulate real-world scenarios, we will define a Product entity using various property types. So, create a file named **Product.cs** within the Entities folder and add the following code:

```
using System.ComponentModel.DataAnnotations.Schema;
namespace EFCoreCodeFirstDemo.Entities
```

```

{
    public class Product
    {
        public int Id { get; set; }
        public string Name { get; set; }
        [Column(TypeName = "decimal(18,2)")]
        public decimal Price { get; set; }
        public int Quantity { get; set; }
        public string Description { get; set; }
        public DateTime CreatedDate { get; set; }
        public string ModifiedBy { get; set; }
        public bool IsAvailable { get; set; }
    }
}

```

Configuring the DbContext

Next, the DbContext class will be set up to manage the database connection and map the Product entity. Modify the **EFCoreDbContext.cs** class as follows:

```

using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Entities
{
    public class EFCoreDbContext : DbContext
    {
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            // Configure the SQL Server connection string
            optionsBuilder.UseSqlServer(@"Server=LAPTOP-
6P5NK25R\SQLSERVER2022DEV;Database=ProductsDB;Trusted_Connection=True;TrustSer
");
            // DbSet representing the Students table
            public DbSet<Product> Products { get; set; }
        }
    }
}

```

Applying Migrations and Updating the Database

After setting up the Product entity and DbContext, apply migrations to create the database schema. Open the Package Manager Console and Execute the **Add-Migration** and **Update-Database** commands. These commands will create the necessary tables in the specified database based on your Product entity.

Bulk Insert Operation Performance Benchmark in Entity Framework Core

Now, we will benchmark the performance of inserting 2,000 Product records using the three methods: Standard EF Core, EFCore.BulkExtensions, and Z.EntityFramework.Extensions.EFCore. So, modify the Program class as follows. Before each insert method, existing data is removed to ensure that each method operates on a clean slate. This ensures that the benchmark measures the insert operation accurately without interference from existing data.

```

using System.Diagnostics;
using EFCoreCodeFirstDemo.Entities;

```

```

namespace EFCoreBulkInsertBenchmark
{
    class Program
    {
        static void Main(string[] args)
        {
            var context = new EFCoreDbContext();
            // Generate the products
            var products = GenerateProducts(2000);
            // Standard EF Core Insert
            var efCoreTime = InsertWithStandardEFCore(products);
            // Z.EntityFramework.Extensions.EFCore Insert
            var zEntityExtensionsTime =
InsertWithZEntityExtensions(products);
            // EFCore.BulkExtensions Insert
            var bulkExtensionsTime =
InsertWithEFCoreBulkExtensions(products);
            // Display the results
            Console.WriteLine("\nBulk Insert Performance Benchmark:");
            Console.WriteLine($"Standard EF Core Insert: {efCoreTime} ms");
            Console.WriteLine($"Z.EntityFramework.Extensions.EFCore Insert:
{zEntityExtensionsTime} ms");
            Console.WriteLine($"EFCore.BulkExtensions Insert:
{bulkExtensionsTime} ms");
        }
        // Generates a list of Product instances.
        static List<Product> GenerateProducts(int count)
        {
            var products = new List<Product>();
            for (int i = 0; i < count; i++)
            {
                products.Add(new Product
                {
                    Name = $"Product_{i}",
                    Price = (decimal)(new Random().NextDouble() * 100),
                    Quantity = 100,
                    Description = $"Product_{i} Description",
                    CreatedDate = DateTime.Now,
                    IsAvailable = true,
                    ModifiedBy = "Admin"
                });
            }
            return products;
        }
        // Inserts products using standard EF Core AddRange and SaveChanges.
        static long InsertWithStandardEFCore(List<Product> products)
        {
            using (var context = new EFCoreDbContext())
            {
                // Clear existing data to ensure a fair benchmark
                context.Products.RemoveRange(context.Products);
            }
        }
    }
}

```

```

        context.SaveChanges();
        var stopwatch = Stopwatch.StartNew();
        context.Products.AddRange(products);
        context.SaveChanges();
        stopwatch.Stop();
        return stopwatch.ElapsedMilliseconds;
    }
}
// Inserts products using EFCore.BulkExtensions BulkInsert.
static long InsertWithEFCoreBulkExtensions(List<Product> products)
{
    // Insert using EFCore.BulkExtensions
    using (var context = new EFCoreDbContext())
    {
        // Clear existing data
        context.Products.RemoveRange(context.Products);
        context.SaveChanges();
        var stopwatch = Stopwatch.StartNew();

        EFCore.BulkExtensions.DbContextBulkExtensions.BulkInsert(context, products);
        stopwatch.Stop();
        return stopwatch.ElapsedMilliseconds;
    }
}
// Inserts products using Z.EntityFramework.Extensions.EFCore
BulkInsert.
static long InsertWithZEntityExtensions(List<Product> products)
{
    using (var context = new EFCoreDbContext())
    {
        // Clear existing data
        context.Products.RemoveRange(context.Products);
        context.SaveChanges();
        var stopwatch = Stopwatch.StartNew();
        DbContextExtensions.BulkInsert(context, products);
        stopwatch.Stop();
        return stopwatch.ElapsedMilliseconds;
    }
}
}
}
}

```

Output:

```

Bulk Insert Performance Benchmark:
Standard EF Core Insert: 404 ms
Z.EntityFramework.Extensions.EFCore Insert: 507 ms
EFCore.BulkExtensions Insert: 166 ms

```

Note: As you can see, the clear winner is EFCore.BulkExtensions. The actual timings will vary based on your system's performance and database configuration. Generally, bulk extension libraries like EFCore.BulkExtensions and Z.EntityFramework.Extensions.EFCore outperforms standard EF Core methods for large datasets.

Bulk Update Performance Benchmarks in Entity Framework Core:

Now, we will perform Bulk Update operations to compare the performance of the different methods. The steps are as follows:

- **Generate and Insert Initial Data:** We first generate and insert 2000 products to ensure there is existing data.
- **Modify Products:** We then modify the properties of these products to prepare them for an update.
- **Execute Updates:** Update the products using three different methods and measure the execution time.

Now, we will evaluate the performance of updating 2,000 Product records using the three methods. So, please modify the Program class as follows:

```
using System.Diagnostics;
using EFCoreCodeFirstDemo.Entities;
namespace EFCoreBulkUpdateBenchmark
{
    class Program
    {
        static void Main(string[] args)
        {
            // Standard EF Core Update
            var efCoreUpdateTime = UpdateWithStandardEFCore();
            // Z.EntityFramework.Extensions.EFCore Update
            var zEntityExtensionsUpdateTime = UpdateWithZEntityExtensions();
            // EFCore.BulkExtensions Update
            var bulkExtensionsUpdateTime = UpdateWithEFCoreBulkExtensions();
            // Display the results
            Console.WriteLine("\nBulk Update Performance Benchmark:");
            Console.WriteLine($"Standard EF Core Update: {efCoreUpdateTime}
ms");
            Console.WriteLine($"Z.EntityFramework.Extensions.EFCore Update:
{zEntityExtensionsUpdateTime} ms");
            Console.WriteLine($"EFCore.BulkExtensions Update:
{bulkExtensionsUpdateTime} ms");
        }
        // Generates a list of Product instances.
        static List<Product> GenerateProducts(int count)
        {
            var products = new List<Product>();
            for (int i = 0; i < count; i++)
            {
                products.Add(new Product
                {
                    Name = $"Product_{i}",
                    Price = (decimal)(new Random().NextDouble() * 100),
                    Quantity = 100,
                    Description = $"Product_{i} Description",
                });
            }
        }
    }
}
```

```

        CreatedDate = DateTime.Now,
        IsAvailable = true,
        ModifiedBy = "Admin"
    });
}
return products;
}
// Inserts products using standard EF Core AddRange and SaveChanges.
static void InsertWithStandardEFCore(List<Product> products)
{
    using (var context = new EFCoreDbContext())
    {
        //Then add new data
        context.Products.AddRange(products);
        context.SaveChanges();
    }
}
// Modify the products for updating
static List<Product> ModifyProductsForUpdate(List<Product> products)
{
    foreach (var product in products)
    {
        product.Price += 10; // Increase price by 10
        product.Quantity += 1;
        product.Description = "Changed";
        product.ModifiedBy = "System";
        product.IsAvailable = !product.IsAvailable; // Flip
availability
    }
    return products;
}
// Updates products using standard EF Core Update and SaveChanges.
static long UpdateWithStandardEFCore()
{
    using (var context = new EFCoreDbContext())
    {
        // Clear existing data
        context.Products.RemoveRange(context.Products);
        context.SaveChanges();
        // Generate initial products and insert them into the
database
        var initialProducts = GenerateProducts(2000);
        // Ensure the data is pre-inserted
        InsertWithStandardEFCore(initialProducts);
        // Modify the products to update them
        var updatedProducts =
ModifyProductsForUpdate(initialProducts);
        var stopwatch = Stopwatch.StartNew();
        context.Products.UpdateRange(updatedProducts);
        context.SaveChanges();
        stopwatch.Stop();
    }
}

```

```

        return stopwatch.ElapsedMilliseconds;
    }
}
// Updates products using EFCore.BulkExtensions BulkUpdate.
static long UpdateWithEFCoreBulkExtensions()
{
    using (var context = new EFCoreDbContext())
    {
        // Clear existing data
        context.Products.RemoveRange(context.Products);
        context.SaveChanges();
        // Generate initial products and insert them into the
database
        var initialProducts = GenerateProducts(2000);
        // Ensure the data is pre-inserted
        InsertWithStandardEFCore(initialProducts);
        // Modify the products to update them
        var updatedProducts =
ModifyProductsForUpdate(initialProducts);
        var stopwatch = Stopwatch.StartNew();
        // Using EFCore.BulkExtensions for bulk update
EFCore.BulkExtensions.DbContextBulkExtensions.BulkUpdate(context,
updatedProducts);
        stopwatch.Stop();
        return stopwatch.ElapsedMilliseconds;
    }
}
// Updates products using Z.EntityFramework.Extensions.EFCore
BulkUpdate.
static long UpdateWithZEntityExtensions()
{
    using (var context = new EFCoreDbContext())
    {
        // Clear existing data
        context.Products.RemoveRange(context.Products);
        context.SaveChanges();
        // Generate initial products and insert them into the
database
        var initialProducts = GenerateProducts(2000);
        // Ensure the data is pre-inserted
        InsertWithStandardEFCore(initialProducts);
        // Modify the products to update them
        var updatedProducts =
ModifyProductsForUpdate(initialProducts);
        var stopwatch = Stopwatch.StartNew();
        // Using Z.EntityFramework.Extensions for bulk update
DbContextExtensions.BulkUpdate(context, updatedProducts);
        stopwatch.Stop();
        return stopwatch.ElapsedMilliseconds;
    }
}

```

```

    }
}
}

```

Output:

```

Bulk Update Performance Benchmark:
Standard EF Core Update: 389 ms
Z.EntityFramework.Extensions.EFCore Update: 278 ms
EFCore.BulkExtensions Update: 290 ms

```

Note: Actual timings will vary based on system performance. Typically, bulk extension libraries provide significantly faster update operations than standard EF Core methods.

Bulk Delete Performance Benchmarks in Entity Framework Core:

Finally, we will assess the performance of deleting 2,000 Product records using the three methods. So, please modify the Program class as follows. Three separate sets of 2,000 products were inserted to ensure that each delete method operated on its own dataset without interference.

```

using System.Diagnostics;
using EFCoreCodeFirstDemo.Entities;
namespace EFCoreBulkDeleteBenchmark
{
    class Program
    {
        static void Main(string[] args)
        {
            // Generate initial products and insert them into the database
            var initialProducts1 = GenerateProducts(2000);
            var initialProducts2 = GenerateProducts(2000);
            var initialProducts3 = GenerateProducts(2000);
            // Ensure the data is pre-inserted
            InsertWithStandardEFCore(initialProducts1);
            InsertWithStandardEFCore(initialProducts2);
            InsertWithStandardEFCore(initialProducts3);
            // Standard EF Core Delete
            var efCoreDeleteTime =
DeleteWithStandardEFCore(initialProducts1);
            // Z.EntityFramework.Extensions.EFCore Delete
            var zEntityExtensionsDeleteTime =
DeleteWithZEntityExtensions(initialProducts2);
            // EFCore.BulkExtensions Delete
            var bulkExtensionsDeleteTime =
DeleteWithEFCoreBulkExtensions(initialProducts3);
            // Display the results
            Console.WriteLine("\nBulk Delete Performance Benchmark:");
            Console.WriteLine($"Standard EF Core Delete: {efCoreDeleteTime}
ms");

```

```

        Console.WriteLine($"Z.EntityFramework.Extensions.EFCore Delete:
{zEntityExtensionsDeleteTime} ms");
        Console.WriteLine($"EFCore.BulkExtensions Delete:
{bulkExtensionsDeleteTime} ms");
    }
    // Generates a list of Product instances.
    static List<Product> GenerateProducts(int count)
    {
        var products = new List<Product>();
        for (int i = 0; i < count; i++)
        {
            products.Add(new Product
            {
                Name = $"Product_{i}",
                Price = (decimal)(new Random().NextDouble() * 100),
                Quantity = 100,
                Description = $"Product_{i} Description",
                CreatedDate = DateTime.Now,
                IsAvailable = true,
                ModifiedBy = "Admin"
            });
        }
        return products;
    }
    // Inserts products using standard EF Core AddRange and SaveChanges.
    static long InsertWithStandardEFCore(List<Product> products)
    {
        using (var context = new EFCoreDbContext())
        {
            var stopwatch = Stopwatch.StartNew();
            context.Products.AddRange(products);
            context.SaveChanges();
            stopwatch.Stop();
            return stopwatch.ElapsedMilliseconds;
        }
    }
    // Deletes products using standard EF Core RemoveRange and
    SaveChanges.
    static long DeleteWithStandardEFCore(List<Product> products)
    {
        using (var context = new EFCoreDbContext())
        {
            var stopwatch = Stopwatch.StartNew();
            context.Products.RemoveRange(products); // Delete using
standard EF Core
            context.SaveChanges();
            stopwatch.Stop();
            return stopwatch.ElapsedMilliseconds;
        }
    }
    // Deletes products using EFCore.BulkExtensions BulkDelete.

```

```

static long DeleteWithEFCoreBulkExtensions(List<Product> products)
{
    using (var context = new EFCoreDbContext())
    {
        var stopwatch = Stopwatch.StartNew();
        // Bulk delete using EFCore.BulkExtensions
        EFCore.BulkExtensions.DbContextBulkExtensions.BulkDelete(context, products);
        stopwatch.Stop();
        return stopwatch.ElapsedMilliseconds;
    }
}
// Deletes products using Z.EntityFramework.Extensions.EFCore
BulkDelete.
static long DeleteWithZEntityExtensions(List<Product> products)
{
    using (var context = new EFCoreDbContext())
    {
        var stopwatch = Stopwatch.StartNew();
        // Bulk delete using Z.EntityFramework.Extensions
        DbContextExtensions.BulkDelete(context, products);
        stopwatch.Stop();
        return stopwatch.ElapsedMilliseconds;
    }
}
}
}
}

```

Output:

```

Bulk Delete Performance Benchmark:
Standard EF Core Delete: 200 ms
Z.EntityFramework.Extensions.EFCore Delete: 485 ms
EFCore.BulkExtensions Delete: 108 ms

```

Note: Actual timings will vary based on system performance. Generally, bulk extension libraries like EFCore.BulkExtensions and Z.EntityFramework.Extensions.EFCore offers faster delete operations than standard EF Core methods.

Benchmark Results Analysis

Based on the benchmark results, here's a summary of performance across different operations:

Operation	Standard EF Core	EFCore.BulkExtensions	Z.EntityFramework.Extensions.EFCore
Insert	Slower	Fastest	Fast
Update	Slower	Fastest	Fast
Delete	Slower	Fastest	Fast

Summary:

- **Bulk Extension Libraries Outperform Standard EF Core:** Both EFCore.BulkExtensions and Z.EntityFramework.Extensions.EFCore provides significant performance improvements for bulk operations compared to standard EF Core methods.
- **EFCore.BulkExtensions Generally Slightly Faster:** In most benchmarks, EFCore.BulkExtensions shows marginally better performance than Z.EntityFramework.Extensions.EFCore. However, the difference is often negligible and may vary based on specific use cases and environments.

Efficient handling of bulk operations is essential for applications dealing with large datasets. This benchmark demonstrates that using bulk extension libraries like EFCore.BulkExtensions and Z.EntityFramework.Extensions.EFCore can lead to performance improvements over standard EF Core methods.

In the next article, I will discuss [Disconnected Entities in Entity Framework Core](#) with Examples. In this article, I explain Performance Benchmarks Between Standard EF Core Bulk Operation vs. EFCore.BulkExtensions vs. Z.EntityFramework.Extensions.EFCore with examples. I hope you enjoyed this article.